
DS N° 1 INFORMATIQUE III

Calculatrice et documents non autorisés

Le langage utilisé doit être le langage C obligatoirement

Attention à la robustesse du programme

Vous pouvez définir autant de fonctions que vous le souhaitez dans chaque exercice.

Il n'y a pas nécessairement l'équivalence : une question $\Leftrightarrow$ une fonction.

Attention : Votre code doit être robuste. Vous n'êtes cependant pas obligés de vérifier l'intégralité des données des structures vues en CM (définir et utiliser la fonction `verifierFile()` par exemple n'est pas obligatoire).

2 pt bonus vous seront cependant accordés si vous faites correctement toutes les vérifications d'intégrité des données des structures utilisées.

Le barème est donné à titre indicatif mais peut être sujet à changement.

Exercice 1 (*Gestion d'une playlist musicale*) (12 pts)

Lire attentivement toutes les questions de l'exercice avant de démarrer.

Dans cet exercice, toute la gestion des structures doit se faire obligatoirement via des pointeurs. De plus il ne doit y avoir aucun tableau statique : quelles que soient leurs tailles, toutes les zones mémoires (tableaux, structures, ...) doivent être allouées dynamiquement, et libérées lorsqu'elles ne sont plus utilisées.

Le but de cet exercice est de développer un programme en C qui gère une playlist musicale. L'utilisateur de l'application confectionne lui-même sa playlist et peut rajouter des musiques à tout moment. Sauf cas particulier, les morceaux seront joués en respectant l'ordre d'ajout de l'utilisateur.

1. Justifier l'utilisation d'une **file dynamique** comme meilleur choix pour implémenter la playlist.
2. Déclarer une structure **Musique** qui devra contenir **uniquement** les informations suivantes :
 - **titre** : le titre de la chanson.
 - **artiste** : le nom de l'artiste.
 - **temps** : la durée de la chanson en secondes.
 - **fav** : une variable indiquant si la musique est dans les favoris de l'utilisateur.
3. Déclarer la ou les structures permettant de gérer au mieux une file de **Musique** (une playlist)
4. Implémenter une fonction/procédure **dureePlaylist(...)** qui va retourner le temps total de lecture (en secondes) d'une playlist passée en paramètre, et va également retourner le nombre de chansons qui constituent cette playlist.
5. Ecrire une procédure **ajoutMusique(...)** qui prend en paramètre une playlist et une **Musique** et qui ajoute cette musique à la fin de la playlist.
6. Ecrire une fonction **jouerMusique(...)** qui va retirer de la file la prochaine **Musique** à jouer, afficher toutes ses informations et la retourner à la fonction appelante.
7. Ecrire une procédure **aleatoire(...)** qui va faire en sorte de mélanger les musiques d'une playlist passée en paramètre.

Le mélange des musiques se fera de la manière suivante :

 - enlever une musique en la défilant simplement de la file.
 - ajouter cette musique dans une autre liste, aléatoirement soit en début, soit en fin.
 - Une fois toutes les musiques traitées, la playlist passée en paramètre contient toutes les musiques mélangées.
8. Ecrire une fonction/procédure **changePosition(...)** qui prend en paramètre une playlist et une **Musique**, et qui décale cette musique d'un cran vers la fin de la liste (cette musique échange sa place avec la musique qui suit).
9. Ecrire une fonction/procédure **favoris(...)** qui prend en paramètre une playlist et qui va conserver uniquement les musiques qui sont marquées comme favorites. Les autres musiques sont retirées de la liste.

Exercice 2 (*Valeurs extremums d'un arbre*) (3.5 pts)

Pour cet exercice, on utilisera la structure d'arbre suivante :

```
typedef struct _arbre{
    float        valeur;
    struct _arbre* fils_G;
    struct _arbre* fils_D;
} Arbre
```

Ecrire une procédure **réursive** qui retournera la valeur minimale **et** la valeur maximale contenues dans cet arbre. On prend l'hypothèse que les valeurs de l'arbre sont comprises entre 0.0 et 1000.0 inclus.

Indiquer de manière détaillée si cette procédure nécessite une initialisation particulière de valeurs, et/ou écrire quelques lignes indiquant comment l'appeler (en supposant qu'un arbre est déjà existant).

Cette procédure ne fera aucun affichage.

Exercice 3 (*Moyenne d'un arbre*) (4.5 pts)

Pour cet exercice, on utilisera la structure d'arbre suivante :

```
typedef struct _arbre{
    int          element;
    struct _arbre* fils_A;
    struct _arbre* fils_B;
    struct _arbre* fils_C;
} Arbre
```

Ecrire une fonction qui va retourner la moyenne réelle du premier **sous-arbre** d'un arbre passé en paramètre. Cette fonction ne prend aucun autre paramètre que l'arbre indiqué précédemment.

Si le premier sous-arbre est inexistant, la fonction doit générer une erreur.

Cette fonction pourra utiliser d'autres fonctions à écrire également si vous en ressentez le besoin.